Lambda frente a Kappa y REST frente a GraphQL: benchmark local sobre marcaciones de asistencia

Mary Luz Nina Palacios
Maestría en Business Intelligence and Data Analytics
Actividad N.º 1: Arquitecturas

***Resumen*—Se implementan y miden, sobre la misma máquina y los mismos datos, dos arquitecturas de ingesta (Lambda y Kappa) y dos de API (REST y GraphQL). El caso de uso son 218 841 marcaciones de asistencia reales y anonimizadas (2019–2021). Kappa procesa el backlog 1,4 veces más rápido y con menos código; Lambda reprocesa el histórico en menos tiempo. REST supera a GraphQL en rendimiento en todos los escenarios con un worker.**

I. Introducción

Una plataforma de BI debe ingerir eventos con baja latencia y, a la vez, poder recalcular sus agregados cuando cambia la lógica. Lambda resuelve esto con dos capas (batch y speed) y Kappa con un solo flujo que se reprocesa desde el log [1]. En el acceso, REST y GraphQL ofrecen el mismo contenido con contratos distintos [2]. El objetivo es medir ambas decisiones en un caso real y concluir cuál conviene y en qué condiciones.

II. Método

Entorno. Un solo equipo: AMD Ryzen 7 5700U with Radeon Graphics, 15 GB de RAM, Windows 11 con Docker 27.3.1 (VM WSL2 con 16 CPU lógicas y 7,5 GiB). PostgreSQL 16.4 (Debian 16.4-1.pgdg120+2), Redpanda v24.2.7 (API de Kafka, topic de 3 particiones) y Python 3.11.10 (FastAPI y Strawberry).

Caso de uso. Marcaciones (entrada, salida y refrigerio) del personal del Ministerio Público, Distrito Fiscal de Puno, de 2019 a junio de 2021: 218 841 eventos, 460 personas y 16 sedes. Los años 2020–2021 tienen muchos menos registros: además del trabajo remoto por la pandemia, para evitar el contagio se usaron otros medios de registro de asistencia que no están en esta fuente. El DNI se reemplaza por un HMAC-SHA256 con sal privada; no se publican datos personales. Se considera tardanza toda entrada posterior a las 08:00 (supuesto). El agregado es, por sede y hora, el número de marcaciones y de tardanzas.

Arquitecturas. *Kappa*: un procesador agrega en `kappa_view_vN`; el reproceso lee el topic desde el offset 0 hacia `v(N+1)` y cambia la vista `kappa_serving`. *Lambda* [3]: un *master sink* guarda cada evento en `master_events`; la capa batch recalcula todo en SQL sobre una tabla A/B y, en una transacción, mueve el *cutoff* y purga la capa speed, que agrega lo posterior por bloques de offset; la consulta une ambas. En las dos, los offsets se guardan en la misma transacción que el agregado (efecto *exactly-once*) y el `event_id` determinista hace idempotentes los reenvíos.

Tabla I
Ingesta (mediana de 3 repeticiones)

Métrica	Lambda	Kappa
Throughput backlog (ev/s)	22 963	32 793
Tiempo de ingesta (s)	9,5	6,7
Latencia e2e p50 (ms)	331	310
Latencia e2e p95 (ms)	581	550
Latencia e2e p99 (ms)	622	577
Reproceso completo (s)	0,61	6,49
CPU procesos, media (%)	120	66
RAM procesos, máx. (MiB)	217	103
CPU PostgreSQL, media (%)	66	32
Líneas de código propias	173	119
Servicios a mantener	3	1
Cuadra con la referencia	sí	sí

Las dos APIs usan el mismo módulo de repositorio y aceptan `arch=lambda|kappa`.

Métricas. Throughput con el backlog completo precargado; latencia de extremo a extremo (commit en la BD menos `produced_at`) a 2 000 ev/s durante 30 s, sin los primeros 5 s y con el batch cada 15 s; tiempo de reproceso; CPU y RAM con `docker stats`; líneas de código; y consistencia exacta frente a la referencia. En la API, con `oha` y un worker por API: escenarios *hourly* (un mes de una sede), *top-sedes* y *POST events* (10 eventos), concurrencia 10 y 50, 10 s de calentamiento y 30 s de medición. Cada escenario se repite 3 veces (API: 3) y se reporta la mediana. Todo se reproduce con `python tasks.py bench`.

III. Resultados

La Tabla I resume la ingesta. Ambas vistas cuadran exactamente con la referencia. Kappa procesa 32 793 ev/s frente a 22 963 ev/s de Lambda, que escribe cada evento dos veces (`master_events` y capa speed). Con carga constante las latencias son del mismo orden (p50 de 310 frente a 331 ms), porque las dominan el sondeo y el tamaño de lote del consumidor. En reproceso la relación se invierte: Lambda recalcula el histórico en 0,61 s con una sola consulta SQL sobre `master_events`, mientras que el replay Kappa tarda 6,5 s porque relee el topic completo.

La Tabla II y la Fig. 1 muestran la API. Con concurrencia 50, REST atiende 7,8 veces más peticiones que GraphQL en *hourly*, 1,4 veces en *top-sedes* y 3,4 veces en la ingesta. La brecha crece con el tamaño de la respuesta: el costo está en la validación y serialización campo a campo de GraphQL, no

Tabla II
API: REQUESTS/S Y LATENCIA EN MS (MEDIANA DE 3 REPETICIONES, TASA DE ERROR MÁXIMA 0,00 %)

			REST			GraphQL		
Escenario	Vista	c	req/s	p50	p99	req/s	p50	p99
hourly	Kappa	10	485	18,4	46,1	64	147,8	225,3
hourly	Kappa	50	496	94,8	159,5	63	785,6	940,6
hourly	Lambda	10	490	19,2	36,7	64	150,3	219,0
hourly	Lambda	50	460	104,6	154,4	60	830,6	1 005,0
top-sedes	Kappa	10	226	44,6	61,3	165	54,9	120,3
top-sedes	Kappa	50	225	222,9	265,9	157	315,5	438,4
top-sedes	Lambda	10	69	145,8	189,4	62	159,2	253,7
top-sedes	Lambda	50	68	726,2	870,7	63	782,7	1 000,2
POST events	–	10	387	20,4	95,1	155	57,8	177,9
POST events	–	50	502	96,0	206,7	147	336,5	491,4

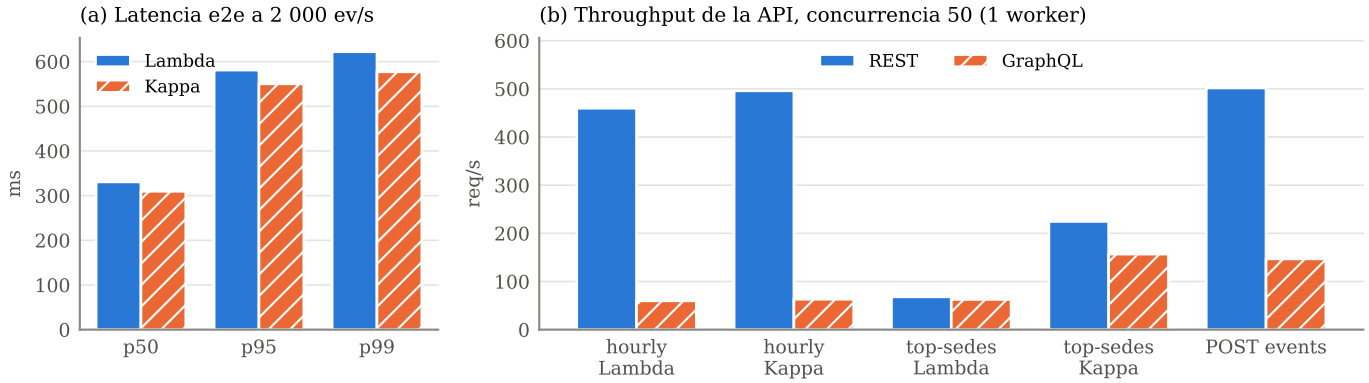

Figura 1. (a) Latencia de extremo a extremo de la ingesta. (b) Throughput de la API por escenario.

en la base de datos. Entre vistas, *top-sedes* es 3,3 veces más rápido sobre Kappa, porque `lambda_serving` debe unir batch y speed en cada consulta.

IV. Discusión

Persico et al. [4] concluyen que Lambda supera a Kappa en análisis de redes sociales en la nube; aquí Kappa gana en ingesta y en simplicidad (119 frente a 173 líneas propias y un servicio frente a tres), pero Lambda gana en reproceso, coincidiendo en que el resultado depende de la carga y del costo de releer el log [1]. En API, la literatura atribuye a GraphQL menos esfuerzo de implementación [5] y respuestas más pequeñas [2]; este benchmark no contradice eso, pero muestra que en Python esa flexibilidad cuesta rendimiento cuando el cliente ya pide exactamente lo que el endpoint REST devuelve.

Amenazas a la validez: un solo nodo y una VM compartida (Redpanda, PostgreSQL, procesadores y cliente compiten por CPU); Python como cuello de botella en procesadores y APIs; el volumen real (218 841 eventos) es pequeño, por eso la carga constante reemite las mismas marcaciones con `produced_at` actual; las marcaciones de 2020–2021 no reflejan toda la asistencia real, porque parte se registró por otros medios; y la regla de tardanza es un supuesto.

V. Conclusión

Para este caso (volumen moderado, una sola lógica de agregación y necesidad de datos frescos) conviene **Kappa**: más throughput, latencia equivalente, menos código y una sola ruta de cálculo. Lambda se justifica si el reproceso frecuente del histórico es crítico o si la lógica batch difiere de la de streaming. Para la API conviene **REST** cuando las consultas son fijas y el rendimiento importa; GraphQL, cuando hay muchos clientes con necesidades de datos distintas. Repositorio: https://github.com/mlnp-dev/bi-lambda-kappa-bench.

Referencias

[1] J. Lin, "The lambda and the Kappa," *IEEE Internet Computing*, vol. 21, no. 5, pp. 60–66, 2017, doi: 10.1109/MIC.2017.3481351.

[2] G. Brito, T. Mombach, and M. T. Valente, "Migrating to GraphQL: A practical assessment," in *Proc. IEEE 26th Int. Conf. Software Analysis, Evolution and Reengineering (SANER)*, 2019, pp. 140–150, doi: 10.1109/SANER.2019.8667986.

[3] M. Kiran, P. Murphy, I. Monga, J. Dugan, and S. S. Baveja, "Lambda architecture for cost-effective batch and speed big data processing," in *Proc. IEEE Int. Conf. Big Data*, 2015, pp. 2785–2792, doi: 10.1109/BigData.2015.7364082.

[4] V. Persico, A. Pescapé, A. Picariello, and G. Sperlí, "Benchmarking big data architectures for social networks data processing using public cloud platforms," *Future Generation Computer Systems*, vol. 89, pp. 98–109, 2018, doi: 10.1016/j.future.2018.05.068.

[5] G. Brito and M. T. Valente, "REST vs GraphQL: A controlled experiment," in *Proc. IEEE Int. Conf. Software Architecture (ICSA)*, 2020, pp. 81–91, doi: 10.1109/ICSA47634.2020.00016.